

Capítulo 4

Backend con Node.js, Express y APIs REST

Desarrollo de Software IX · Código 1493 · Módulo III

Propósito del capítulo

Explicar la teoría necesaria para construir servicios backend con Node.js y Express: módulos, servidores HTTP, rutas, middlewares, controladores, manejo de errores y documentación REST.

Tabla de contenido

1. Panorama del capítulo	3
2. HTTP: el protocolo que sostiene la web	3
3. Node.js y el modelo de entrada/salida	4
4. El sistema de módulos	5
5. Servidores HTTP: del módulo nativo a Express	5
6. Express como framework HTTP	6
7. REST como estilo arquitectónico	7
8. Estructura backend recomendada	8
9. Middlewares: el patrón cadena de responsabilidad	8
10. Validación autoritativa en el servidor	9
11. Manejo de errores centralizado	9
12. Documentación de APIs	10
13. Más allá de Express: frameworks sobre estándares web	11
14. Caso guía: API de reservas de laboratorios	11
15. Actividades sugeridas	11
16. Rúbrica breve de evaluación	12
17. Cierre	12

1. Panorama del capítulo

El backend es la capa que protege y coordina la lógica de la aplicación. Recibe solicitudes, valida datos, ejecuta reglas de negocio, consulta la base de datos y devuelve respuestas. Para construirlo con criterio no basta con memorizar la sintaxis de Express: hay que entender el protocolo HTTP sobre el que viaja toda la comunicación, el modelo de ejecución de Node.js que determina qué tipo de servidor conviene escribir, y los principios del estilo arquitectónico REST que dan forma al contrato entre cliente y servidor. Este capítulo recorre esa cadena completa: del protocolo al framework, y del framework al diseño.

Idea central

Una API no es un conjunto de rutas sueltas. Es un contrato entre frontend y backend: define recursos, métodos, entradas, salidas, errores y permisos. Cada decisión técnica de este capítulo —elegir un método HTTP, un código de estado, un nombre de ruta— es una cláusula de ese contrato.

2. HTTP: el protocolo que sostiene la web

HTTP (HyperText Transfer Protocol) es un protocolo de solicitud y respuesta: el cliente envía una solicitud, el servidor la procesa y devuelve exactamente una respuesta. Es, además, un protocolo sin estado: el servidor no recuerda nada entre solicitudes, y cada una debe contener toda la información necesaria para procesarla. Esta propiedad, que al inicio parece una limitación, es la que permite escalar servicios distribuyendo solicitudes entre múltiples servidores. Toda solicitud tiene tres partes: una línea inicial con método, ruta y versión del protocolo; encabezados que describen el mensaje (formato, tamaño, credenciales); y un cuerpo opcional con datos. La respuesta sigue la misma estructura, pero su línea inicial declara un código de estado que resume el resultado. Entender esta anatomía es esencial porque Express no inventa nada: solo ofrece una interfaz cómoda sobre estas piezas.

```
GET /api/v1/productos/7 HTTP/1.1
Host: api.ejemplo.com
Accept: application/json
```

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{ "id": 7, "nombre": "Teclado", "precio": 25.50 }
```

2.1 Métodos HTTP y su semántica

Cada método HTTP expresa una intención. Dos propiedades teóricas distinguen su comportamiento. Un método es seguro cuando no modifica el estado del servidor: solo lee. Un método es idempotente cuando ejecutarlo varias veces produce el mismo efecto que ejecutarlo una sola vez. La idempotencia no es un detalle académico: si una red falla y el cliente no sabe si su solicitud llegó, solo puede reintentarla con tranquilidad si el método es idempotente. Reintentar un `PUT` deja el recurso en el mismo estado final; reintentar un `POST` puede crear el recurso dos veces. Por eso los clientes HTTP, los proxies y las pasarelas de pago tratan cada método de forma distinta.

Método	Semántica	¿Seguro?	¿Idempotente?
GET	Recuperar la representación de un recurso. Nunca debe modificar datos.	Sí	Sí
POST	Crear un recurso subordinado o iniciar un proceso. Cada envío puede producir un efecto nuevo.	No	No
PUT	Reemplazar un recurso completo con la representación enviada.	No	Sí
PATCH	Aplicar una modificación parcial al recurso.	No	No (en general)
DELETE	Eliminar el recurso identificado por la URL.	No	Sí

2.2 Códigos de estado por familia

Los códigos de estado son la parte del contrato que comunica resultados sin necesidad de leer el cuerpo. Se agrupan en familias según el primer dígito, y elegirlos bien es una decisión de diseño: un backend que responde 200 ante un error obliga al frontend a adivinar inspeccionando el cuerpo, y rompe herramientas estándar como cachés, monitores y clientes HTTP que toman decisiones automáticas según la familia.

Familia	Significado	Ejemplos representativos
1xx	Informativo: la solicitud continúa en proceso.	100 Continue, 101 Switching Protocols.
2xx	Éxito: el servidor procesó la solicitud correctamente.	200 OK, 201 Created, 204 No Content.
3xx	Redirección: el recurso está en otro lugar o no cambió.	301 Moved Permanently, 304 Not Modified.
4xx	Error del cliente: la solicitud es inválida o no permitida.	400, 401, 403, 404, 409, 422.
5xx	Error del servidor: la solicitud era válida pero el servidor falló.	500 Internal Server Error, 503 Service Unavailable.

3. Node.js y el modelo de entrada/salida

Node.js es un entorno de ejecución que permite correr JavaScript fuera del navegador. Combina el motor V8 (que compila y ejecuta el lenguaje) con la librería libuv (que gestiona operaciones de entrada/salida del sistema operativo: red, disco, temporizadores). Su aporte conceptual no es el lenguaje sino el modelo de ejecución: en lugar de dedicar un hilo del sistema operativo a cada conexión, como hacían los servidores tradicionales, Node atiende todas las conexiones desde un único hilo principal coordinado por un bucle de eventos.

El event loop funciona como un despachador: toma una tarea pendiente, la ejecuta hasta terminar y pasa a la siguiente. Cuando el código solicita una operación lenta —leer un archivo, consultar MongoDB, llamar a una API externa— Node delega esa operación al sistema operativo, registra una función de retorno (callback o promesa) y sigue atendiendo otras solicitudes. Cuando la operación termina, el resultado vuelve a la cola y el event loop ejecuta la continuación. Esto se llama entrada/salida no bloqueante: el hilo nunca espera de brazos cruzados.

Este modelo es especialmente adecuado para APIs porque el trabajo típico de un backend web es intensivo en entrada/salida, no en cálculo: la mayor parte del tiempo de cada solicitud se gasta esperando a la base de datos o a otros servicios. Un solo hilo no bloqueante puede mantener miles de conexiones concurrentes con poca memoria. La contrapartida es directa: como hay un único hilo para el código JavaScript, cualquier tarea de cálculo pesado (un bucle enorme, un procesamiento de imagen síncrono) bloquea el event loop y congela el servidor entero para todos los usuarios. De ahí la regla práctica del desarrollo en Node: nunca bloquear el event loop. Las palabras clave `async/await` son la sintaxis moderna para expresar este modelo sin anidar callbacks.

```
// Lectura no bloqueante: el hilo queda libre mientras el disco responde.
const { readFile } = require('node:fs/promises');

async function cargarConfiguracion() {
  try {
    const texto = await readFile('config.json', 'utf8');
    return JSON.parse(texto);
  } catch (error) {
    console.error('No se pudo leer la configuración:', error.message);
    throw error;
  }
}
```

4. El sistema de módulos

Un módulo es una unidad de código con interfaz explícita: declara qué expone y qué oculta. Los módulos existen porque los programas crecen, y sin fronteras claras cualquier función puede depender de cualquier variable, lo que hace imposible razonar sobre el sistema o probarlo por partes. Node.js popularizó dos sistemas de módulos. CommonJS (`require` / `module.exports`) nació con Node y carga módulos de forma síncrona y dinámica. ES Modules (`import` / `export`) es el estándar oficial del lenguaje: sus importaciones son estáticas, lo que permite a las herramientas analizar dependencias sin ejecutar el código. Sobre este sistema se apoya npm, el gestor de paquetes: `package.json` declara las dependencias del proyecto y los scripts de ejecución, y el campo `type: "module"` indica a Node qué sistema usar por defecto.

Aspecto	CommonJS	ES Modules
Sintaxis	<code>const x = require('mod') / module.exports</code>	<code>import x from 'mod' / export</code>
Momento de carga	Dinámica, en tiempo de ejecución.	Estática, analizable antes de ejecutar.
Origen	Convención histórica de Node.js.	Estándar ECMAScript del lenguaje.
Activación en Node	Por defecto en archivos .js (sin <code>type module</code>).	Con <code>type: "module"</code> o extensión .mjs.

```
// CommonJS (estilo histórico de Node)
const express = require('express');
module.exports = { crearServidor };

// ES Modules (estándar del lenguaje; "type": "module" en package.json)
import express from 'express';
export function crearServidor() { /* ... */ }
```

5. Servidores HTTP: del módulo nativo a Express

Node incluye el módulo `node:http`, capaz de crear un servidor completo sin instalar nada. Trabajar con él una vez es un ejercicio valioso porque revela todo lo que un framework hace por nosotros: con el módulo nativo, distinguir rutas exige comparar cadenas manualmente, leer un cuerpo JSON exige acumular fragmentos del stream, y cada respuesta exige escribir encabezados a mano. El ejemplo siguiente muestra ese costo: una sola ruta ya requiere lógica condicional explícita.

```
const http = require('node:http');

const server = http.createServer((req, res) => {
  if (req.method === 'GET' && req.url === '/salud') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    return res.end(JSON.stringify({ estado: 'ok' }));
  }
  res.writeHead(404, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ error: 'Ruta no encontrada' }));
});

server.listen(3000);
```

6. Express como framework HTTP

Express es una capa delgada sobre el módulo nativo que resuelve los problemas repetitivos: enrutamiento declarativo (asociar método y patrón de URL con una función), análisis del cuerpo JSON, lectura de parámetros y un mecanismo de middlewares para componer comportamiento. Su filosofía es minimalista y poco opinada: no impone estructura de carpetas ni capas. Esa flexibilidad es su mayor virtud y su mayor riesgo: sin disciplina, el servidor se convierte en un archivo gigante difícil de mantener. Por eso este capítulo acompaña el framework con convenciones de organización.

Concepto	Función	Ejemplo
Ruta	Asocia método y URL con una función.	GET /api/productos
Request	Representa datos enviados por el cliente.	params, query, body, headers
Response	Permite devolver estado y datos.	res.status(201).json(...)
Middleware	Procesa la solicitud antes o después de una ruta.	auth, logger, validator
Router	Agrupar rutas relacionadas.	router.use('/users', userRoutes)

6.1 Rutas GET, POST, PUT y DELETE

Una ruta de Express recibe la solicitud por tres canales distintos, y cada uno tiene un propósito semántico. Los parámetros de ruta (`req.params`) identifican un recurso concreto: el `:id` en `/productos/:id`. Los parámetros de consulta (`req.query`) modifican una lectura: filtros, paginación, ordenamiento. El cuerpo (`req.body`) transporta la representación del recurso en operaciones de escritura. Mezclarlos —por ejemplo, enviar el identificador del recurso a eliminar dentro del cuerpo de un GET— rompe las expectativas de cualquier consumidor de la API. Un `Router` agrupa las operaciones de un mismo recurso y permite montarlas bajo un prefijo común.

```
const router = require('express').Router();

router.get('/', listarProductos);           // GET    /api/v1/productos
router.get('/:id', obtenerProducto);       // GET    /api/v1/productos/7
router.post('/', crearProducto);           // POST   /api/v1/productos
router.put('/:id', actualizarProducto);    // PUT    /api/v1/productos/7
router.delete('/:id', eliminarProducto);   // DELETE /api/v1/productos/7

module.exports = router;
```

7. REST como estilo arquitectónico

REST (Representational State Transfer) no es una librería ni un protocolo: es un estilo arquitectónico formulado por Roy Fielding en su tesis doctoral del año 2000, a partir del análisis de por qué la web logró escalar globalmente. Su unidad de diseño es el recurso: cualquier cosa con identidad que valga la pena nombrar (un producto, una reserva, un usuario), identificada por una URL estable. El cliente nunca toca el recurso directamente: intercambia representaciones de él —típicamente JSON— y el servidor decide cómo almacenarlo. Los métodos HTTP forman la interfaz uniforme: el mismo verbo significa lo mismo sobre cualquier recurso, lo que permite a un consumidor predecir el comportamiento de una API sin leer su código.

- Cliente-servidor: interfaz y datos evolucionan por separado; el frontend puede cambiar sin tocar el backend y viceversa.
- Sin estado (stateless): cada solicitud lleva toda la información necesaria, incluida la identidad del usuario; el servidor no guarda sesión en memoria, lo que permite repartir solicitudes entre varias instancias.
- Cacheable: las respuestas declaran si pueden reutilizarse, y los métodos seguros habilitan cachés intermedias sin riesgo.
- Interfaz uniforme: recursos identificados por URL, manipulados mediante representaciones y operados con métodos de semántica fija.
- Sistema en capas: pueden existir proxies, balanceadores y pasarelas entre cliente y servidor sin que ninguno lo note.

7.1 Diseño de rutas y versionado

Las URLs de una API REST nombran recursos con sustantivos, normalmente en plural, y dejan que el método exprese la acción: la operación 'borrar producto' no es una ruta `/borrarProducto` sino `DELETE /productos/:id`. Las relaciones se expresan anidando con moderación (`/labs/:id/reservas`) y los filtros viajan como query (`/reservas?estado=pendiente`). El versionado existe porque el contrato cambia: cuando una modificación rompe a los consumidores existentes —renombrar un campo, eliminar una ruta— se publica una nueva versión y se mantiene la anterior durante una transición. La convención más simple y visible es el prefijo de ruta `/api/v1/`, que este curso adopta.

Diseño problemático	Diseño REST	Razón
GET /getProductos	GET /api/v1/productos	El verbo ya lo aporta el método HTTP; la URL nombra el recurso.
POST /producto/borrar/5	DELETE /api/v1/productos/5	DELETE expresa la intención y es idempotente ante reintentos.

Diseño problemático	Diseño REST	Razón
GET /crearReserva?lab=2	POST /api/v1/reservas	GET es seguro por contrato: jamás debe crear ni modificar.
Rutas sin versión	Prefijo /api/v1/	Permite evolucionar el contrato sin romper clientes existentes.

8. Estructura backend recomendada

Como Express no impone organización, el equipo debe imponerla. La estructura por capas que sigue separa tres preocupaciones distintas: el contrato HTTP (rutas), la traducción entre HTTP y dominio (controladores) y las reglas de negocio (servicios). El beneficio es teórico y práctico a la vez: cada capa puede cambiarse o probarse sin arrastrar a las demás.

Carpeta	Contenido	Razón
routes	Definición de endpoints y middlewares por recurso.	Mantiene visible el contrato HTTP.
controllers	Funciones que reciben request y devuelven response.	Evita mezclar rutas con lógica extensa.
services	Reglas de negocio reutilizables.	Facilita pruebas y cambios.
models	Modelos Mongoose o entidades de datos.	Centraliza estructura y validaciones.
middlewares	Autenticación, errores, validación, logging.	Reutiliza comportamiento transversal.
config	Conexión a base de datos y variables.	Separa configuración del dominio.

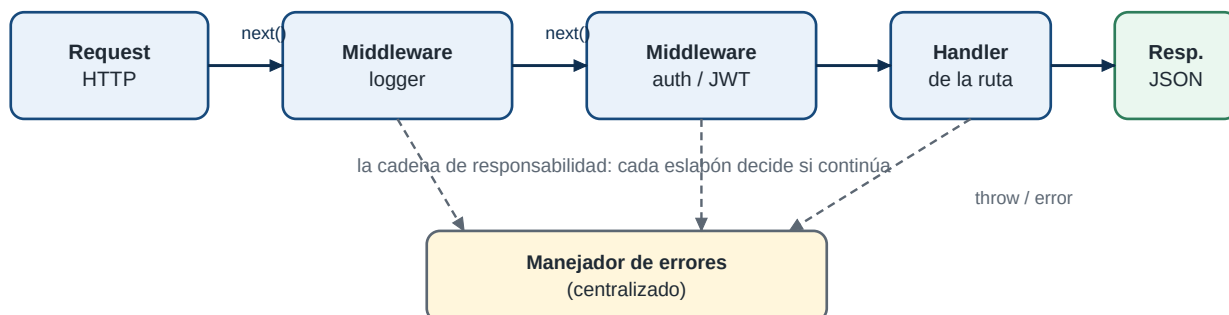
9. Middlewares: el patrón cadena de responsabilidad

El middleware es la idea más importante de Express y corresponde a un patrón de diseño clásico: la cadena de responsabilidad. Cada solicitud atraviesa una secuencia ordenada de funciones; cada una puede examinarla, transformarla, responder de inmediato o ceder el control a la siguiente invocando `next()`. Este patrón resuelve un problema real: hay comportamiento transversal —registrar solicitudes, leer el cuerpo JSON, verificar el token, validar datos— que pertenece a muchas rutas a la vez, y duplicarlo en cada controlador sería inmanejable. Dos consecuencias prácticas se derivan del modelo: el orden de registro importa (un middleware de autenticación colocado después de las rutas no protege nada) y olvidar `next()` deja la solicitud colgada para siempre, porque ningún eslabón posterior se ejecuta.

```
function registrarSolicitud(req, res, next) {
  console.log(`${req.method} ${req.originalUrl}`);
  next(); // cede el control al siguiente eslabón de la cadena
}

app.use(express.json());           // 1. interpreta cuerpos JSON
app.use(registrarSolicitud);       // 2. registra cada solicitud
app.use('/api/v1/productos', productosRouter); // 3. rutas del recurso
```


Tipo	Ejemplo	Propósito
De aplicación	<code>express.json()</code> , logger propio	Se aplica a todas las solicitudes registradas con <code>app.use</code> .
De router	<code>router.use(verificarToken)</code>	Afecta solo a las rutas de un recurso o grupo.
De terceros	<code>cors</code> , <code>morgan</code> , <code>helmet</code>	Funcionalidad transversal mantenida por la comunidad.
De error	Función con cuatro parámetros (<code>err</code> , <code>req</code> , <code>res</code> , <code>next</code>)	Captura fallos de toda la cadena en un único lugar.



10. Validación autoritativa en el servidor

El frontend puede validar formularios para mejorar la experiencia, pero esa validación es solo una cortesía: cualquier persona puede enviar solicitudes directamente con Postman, curl o un script, saltándose la interfaz por completo. Por eso la validación del servidor es la autoritativa: es la única barrera que el cliente no puede evadir. Validar significa comprobar presencia, tipo, formato y rango de cada campo antes de tocar la base de datos, y rechazar lo inválido con un 400 y un mensaje que indique exactamente qué campo falló. Un backend que confía en datos del cliente no tiene un bug: tiene una vulnerabilidad.

Regla práctica

Todo dato que llega por params, query, body o headers es entrada no confiable. La pregunta de diseño no es '¿el frontend ya validó esto?' sino '¿qué pasa si alguien envía cualquier cosa aquí a propósito?'.

11. Manejo de errores centralizado

Un backend profesional no debe fallar con mensajes crípticos ni devolver siempre 500. Los errores deben clasificarse —validación, autenticación, autorización, recurso no encontrado, conflicto, error interno— porque cada clase comunica al cliente una acción distinta: corregir los datos, iniciar sesión, desistir o reintentar más tarde. La centralización es el complemento: en lugar de repetir bloques de respuesta de error en cada controlador, los controladores lanzan o delegan errores y un único middleware final los traduce a código HTTP y cuerpo JSON consistentes. Así el formato de error es uniforme en toda la API y existe un solo lugar donde registrar y depurar fallos.

Caso	Código HTTP	Ejemplo
Datos inválidos	400	Falta nombre o formato incorrecto.
No autenticado	401	Token ausente o inválido.
No autorizado	403	Usuario no tiene permiso.
No encontrado	404	Solicitud inexistente.
Conflicto	409	Correo ya registrado.
Error interno	500	Fallo no esperado controlado por middleware.

```

class AppError extends Error {
  constructor(mensaje, statusCode) {
    super(mensaje);
    this.statusCode = statusCode;
  }
}

// Middleware de error: Express lo reconoce por sus cuatro parámetros.
app.use((err, req, res, next) => {
  const codigo = err.statusCode || 500;
  console.error(err); // registro interno completo
  res.status(codigo).json({ error: err.message || 'Error interno' });
});

```

La depuración acompaña al manejo de errores. Las herramientas básicas son tres: registros (logs) con contexto —método, ruta, código devuelto y duración—, que convierten un fallo reportado por un usuario en un rastro verificable; la inspección de solicitudes con Postman o curl, que aísla si el problema está en el backend o en quien lo consume; y el depurador integrado de Node o del editor, que permite pausar la ejecución y examinar variables en vez de sembrar `console.log` a ciegas. Una distinción importante: el registro interno puede ser detallado, pero la respuesta al cliente nunca debe exponer trazas de pila ni detalles de infraestructura, porque esa información facilita ataques.

12. Documentación de APIs

Una API existe para que otros la consuman, y su documentación es la materialización del contrato. La documentación mínima de cada endpoint incluye: ruta, método, propósito, parámetros, cuerpo esperado, respuesta exitosa y errores posibles. Las dos formas estándar de la industria son las colecciones de Postman —solicitudes de ejemplo ejecutables, que documentan y permiten probar al mismo tiempo— y la especificación OpenAPI, un documento estructurado (YAML o JSON) que describe la API de forma legible por máquinas. El valor de OpenAPI es precisamente ese carácter formal: de una misma especificación se generan portales de documentación interactivos, clientes y pruebas automatizadas, y el contrato deja de vivir solo en la memoria del equipo. Para un proyecto de curso, un README riguroso o una colección de Postman compartida cumplen el mismo rol a menor escala.

Criterio evaluable

Si el frontend no puede usar una ruta sin preguntarle al backend qué enviar o qué esperar, la API no está suficientemente documentada.

13. Más allá de Express: frameworks sobre estándares web

Express definió en 2010 las abstracciones que este capítulo estudia —rutas, middlewares, request y response— y casi todos los frameworks posteriores las heredan. La generación moderna, con Hono como ejemplo destacado, conserva ese modelo conceptual pero lo construye sobre los estándares web (`Request`, `Response` y `fetch`, los mismos objetos que existen en el navegador) en lugar de sobre las clases propias de Node. La consecuencia es portabilidad: el mismo código corre en Node, Bun, Deno o plataformas de borde sin adaptadores. Para efectos de este capítulo basta la idea puente: quien domina rutas, middlewares y manejo de errores en Express ya domina los conceptos de Hono; cambia la superficie de la API, no la teoría. El material específico de Hono se trabaja por separado en el curso.

14. Caso guía: API de reservas de laboratorios

El caso guía del curso —reserva de laboratorios de la facultad— permite aterrizar cada concepto del capítulo. Los recursos son identificables (usuarios, laboratorios, reservas), las operaciones se reparten naturalmente entre métodos HTTP y los errores previsibles cubren todas las familias estudiadas: datos inválidos, credenciales incorrectas, permisos insuficientes y conflictos de horario. La tabla resume el contrato inicial; el estudiante debe poder justificar el método, el código de éxito y los códigos de error de cada fila usando la teoría de las secciones anteriores.

Ruta	Responsabilidad	Errores a prever
POST /api/auth/login	Validar credenciales y devolver token.	Credenciales inválidas, usuario inactivo.
GET /api/labs	Listar laboratorios disponibles.	Fallo de base de datos.
POST /api/reservas	Crear reserva si el horario está libre.	Datos inválidos, choque de horario.
PUT /api/reservas/:id/estado	Aprobar o rechazar reserva.	No autorizado, reserva inexistente.

15. Actividades sugeridas

- Diseñar la tabla de endpoints del recurso principal del proyecto, justificando método y códigos de estado de cada operación.
- Implementar un servidor con el módulo nativo `node:http` y luego reescribirlo en Express, documentando qué simplificó el framework.
- Separar rutas, controladores y servicios en un backend Express con al menos un recurso CRUD completo.
- Escribir un middleware propio (registro de solicitudes o verificación de un encabezado) y razonar dónde debe ubicarse en la cadena.
- Implementar el middleware de manejo de errores centralizado y provocar deliberadamente respuestas 400, 404, 409 y 500 controlado.

- Documentar los endpoints en README o en una colección de Postman y pedir a otro equipo que consuma la API solo con esa documentación.

16. Rúbrica breve de evaluación

Criterio	Peso orientativo	Evidencia de logro
Diseño de recursos y rutas	20%	URLs con sustantivos, versionadas, sin verbos; métodos coherentes con su semántica.
Códigos de estado y contrato	20%	Respuestas 2xx/4xx/5xx correctas según el caso; formato JSON consistente.
Estructura y middlewares	20%	Capas separadas (rutas, controladores, servicios); middlewares ordenados y justificados.
Validación y manejo de errores	25%	Validación autoritativa en el servidor y middleware de error centralizado funcionando.
Documentación de la API	15%	README, Postman u OpenAPI suficiente para consumir la API sin preguntar al autor.

17. Cierre

El backend expresa la lógica confiable del sistema, y este capítulo mostró que sus decisiones no son arbitrarias: la semántica de los métodos HTTP protege contra reintentos, el modelo de event loop explica qué cargas tolera Node, las restricciones REST permiten que cliente y servidor evolucionen por separado, y la cadena de middlewares organiza el comportamiento transversal sin duplicarlo. Cuando estas ideas se aplican con disciplina, el frontend consume la API con claridad, la base de datos queda protegida detrás de validación autoritativa y el proyecto se vuelve mantenible. Los capítulos siguientes construyen sobre esta base: la seguridad con JWT extiende la cadena de middlewares, y la persistencia con MongoDB ocupa la capa de modelos que aquí quedó delimitada.